

Session 12: Object-Oriented Linux Environment & OOPS Concepts

Detailed Theory

1. Object-Oriented Linux Environment

- Linux as an operating system is primarily **process-based** and **command-line driven**, but many Linux tools and environments support object-oriented paradigms indirectly.
- **Object-oriented programming (OOP)** concepts are often used in scripting and programming on Linux, especially with languages like Python, C++, Java, and shell scripting extensions.
- **Linux filesystem** can be seen as an object hierarchy: files and directories behave like objects with attributes (permissions, size, timestamps) and methods (read, write, execute).
- Common OOP languages available in Linux include Python, Java, C++, Ruby, and Perl, all supporting classes and objects.

- Linux supports **shared libraries** (.so files) which can encapsulate reusable code and objects.
 - **Linux kernel** itself uses some OOP-like structures (e.g., struct inheritance in C, polymorphism through function pointers).
 - Tools like **GObject** (GNOME Object system) provide object-oriented APIs in C for Linux applications.
 - Developers working in Linux environments often use OOP to build modular, reusable, and maintainable software.
-

2. Classes, Objects, and OOPS Concepts

- **Object-Oriented Programming (OOP)** is a programming paradigm based on the concept of **objects** that contain data and behavior.
- Core OOP concepts include:

a. Class:

- Blueprint or template for creating objects.
- Defines attributes (data members) and methods (functions).

- Example:

python

CopyEdit

```
class Car: def __init__(self, make, model): self.make = make self.model = model
```

b. Object:

- An instance of a class.

- Has state (attributes) and behavior (methods).

- Example:

```
python
```

```
CopyEdit
```

```
my_car = Car("Toyota", "Camry")
```

c. Encapsulation:

- Bundling data and methods inside a class.
- Restricts direct access to some components (private/protected variables).

- Achieved using access modifiers or naming conventions (e.g., underscore prefix in Python).

d. Inheritance:

- Mechanism to create new classes (child classes) from existing classes (parent classes), inheriting attributes and methods.

- Supports code reuse and hierarchical relationships.

- Example:

python

CopyEdit

```
class ElectricCar(Car):  
    def __init__(self, make, model, battery_size):  
        super().__init__(make, model)  
        self.battery_size = battery_size
```

e. Polymorphism:

- Ability to use a single interface to represent different data types.
- Method overriding allows child classes to provide specific implementations.
- Example:

python

CopyEdit

```
class Animal: def speak(self): pass class Dog(Animal): def speak(self): return "Woof" class Cat(Animal): def speak(self): return "Meow"
```

f. Abstraction:

-

Hiding complex implementation details and showing only essential features.

-

Achieved using abstract classes or interfaces (Python's abc module).

-

Constructor (`__init__`) method: special method to initialize new objects.

-

Self parameter: refers to the current instance of the class. Required in instance methods.

3. Classes and Objects in Linux Scripting

-

Python is commonly used for OOP in Linux scripting due to its simplicity and power.

-

Shell scripting itself is not inherently object-oriented but can call object-oriented programs.

-

In Linux development, OOP is crucial for creating GUI apps, system utilities, and network applications.